

3. API DOCUMENTATION

SAMS (Student Attendance Management System) - Complete REST API Reference

Table of Contents

1. [API Overview](#)
 2. [Authentication](#)
 3. [Base URLs](#)
 4. [Request/Response Format](#)
 5. [Student Endpoints](#)
 6. [Teacher Endpoints](#)
 7. [Admin Endpoints](#)
 8. [Common Endpoints](#)
 9. [Error Handling](#)
 10. [Status Codes](#)
-

API Overview

API Type: RESTful JSON **API Version:** 2.0 **Authentication:** Session-based **Content-Type:** application/json
CORS: Enabled for cross-origin requests

Core Features: - User authentication and authorization - Student attendance marking with GPS + Face verification - Teacher class management - Admin user and system management - Push notifications via FCM - Real-time reports and analytics

Authentication

Login Endpoint

POST /api/login (Public)

Request:

```
{
  "email": "student@example.com",
  "password": "password123",
  "device_token": "fcm_registration_token" // Optional
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "user": {
```

```

    "id": 123,
    "full_name": "John Doe",
    "email": "student@example.com",
    "role": "student",
    "phone": "+91-9876543210"
  },
  "session_id": "abc123def456ghi789",
  "profile": {
    "id": 1,
    "roll_number": "CSE-001-2024",
    "department_id": 2,
    "department_name": "Computer Science",
    "semester": 2,
    "section": "A",
    "batch_year": 2024,
    "face_registered": true
  }
}
}

```

Response (401 Unauthorized):

```

{
  "success": false,
  "message": "Invalid email or password"
}

```

Authentication Headers: After login, all subsequent requests must include:

Cookie: PHPSESSID=abc123def456ghi789

Base URLs

Development: <http://localhost:8000/api/>
 Heroku: <https://sams-backend-xxx.herokuapp.com/api/>
 Azure: <https://sams-backend-xxx.azurewebsites.net/api/>
 Docker: <http://localhost:9000/api/>

Request/Response Format

Standard Request Headers

Content-Type: application/json
 Accept: application/json
 User-Agent: AndroidClient/1.0
 Accept-Language: en-US

Standard Response Format

Success Response:

```

{
  "success": true,
  "message": "Operation completed",
  "data": { /* actual data */ }
}

```

Error Response:

```
{
  "success": false,
  "message": "Error description",
  "errors": {
    "field_name": "Field error message"
  }
}
```

HTTP Methods

- **GET:** Retrieve data (no body)
 - **POST:** Create data or perform action (with body)
 - **PUT:** Update entire resource (with body)
 - **DELETE:** Remove resource (no body)
 - **PATCH:** Partial update (with body)
-

Student Endpoints

1. Student Dashboard

GET /api/student/dashboard

Headers:

Authorization: Bearer session_token
X-Role: student

Response (200 OK):

```
{
  "success": true,
  "data": {
    "profile": {
      "id": 1,
      "user_id": 123,
      "full_name": "John Doe",
      "email": "john@example.com",
      "phone": "+91-9876543210",
      "roll_number": "CSE-001-2024",
      "department_id": 2,
      "department_name": "Computer Science",
      "semester": 2,
      "section": "A",
      "batch_year": 2024,
      "face_registered": true,
      "profile_image": "https://sams.edu/uploads/123.jpg"
    },
    "attendance": {
      "total_classes": 45,
      "attended": 42,
      "percentage": 93.33,
      "status": "Good"
    },
    "subject_wise": [
      {
        "subject_id": 5,
```

```

    "subject_name": "Data Structures",
    "subject_code": "CS201",
    "total_classes": 15,
    "attended": 14,
    "percentage": 93.33
  },
  {
    "subject_id": 6,
    "subject_name": "Algorithms",
    "subject_code": "CS202",
    "total_classes": 15,
    "attended": 14,
    "percentage": 93.33
  }
],
"recent_attendance": [
  {
    "attendance_id": 1001,
    "schedule_id": 50,
    "subject_name": "Data Structures",
    "subject_code": "CS201",
    "teacher_name": "Prof. Smith",
    "date": "2024-01-15",
    "time": "09:00:00",
    "status": "present",
    "verification_status": "success",
    "face_confidence": 95.2,
    "distance_meters": 5.3
  }
],
"low_attendance_subjects": [
  {
    "subject_name": "Web Development",
    "percentage": 60.0,
    "classes_needed": 5
  }
],
"today_schedule": [
  {
    "schedule_id": 50,
    "subject_name": "Data Structures",
    "subject_code": "CS201",
    "teacher_name": "Prof. Smith",
    "start_time": "09:00",
    "end_time": "10:00",
    "classroom": "101",
    "building": "Building-A"
  }
],
"active_session": {
  "schedule_id": 50,
  "subject_code": "CS201",
  "subject_name": "Data Structures",
  "teacher_name": "Prof. Smith",
  "started_at": "2024-01-15T09:00:00Z"
}
}

```

2. Register Face

POST /api/student/register-face

Request:

```
{
  "face_embedding": "base64_encoded_facial_vector",
  "face_photo": "base64_encoded_face_image"
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Face registered successfully",
  "data": {
    "student_id": 1,
    "face_registered": true,
    "registration_date": "2024-01-15T10:30:00Z"
  }
}
```

3. Mark Attendance

POST /api/student/mark-attendance

Request:

```
{
  "schedule_id": 50,
  "latitude": 28.6139,
  "longitude": 77.2090,
  "face_confidence": 92.5,
  "face_embedding": "base64_encoded_facial_vector"
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Attendance marked successfully",
  "data": {
    "attendance_id": 1001,
    "status": "present",
    "verification_status": "success",
    "face_confidence": 92.5,
    "distance_meters": 8.2,
    "timestamp": "2024-01-15T09:05:00Z"
  }
}
```

Response (400 Bad Request - Out of Range):

```
{
  "success": false,
  "message": "GPS location out of range",
  "data": {
    "distance_meters": 150.5,
    "required_radius": 50
  }
}
```

4. Attendance History

GET /api/student/attendance-history?page=1&limit=20&month=01&year=2024

Query Parameters: - page (int): Page number for pagination (default: 1) - limit (int): Records per page (default: 20, max: 100) - month (int): Filter by month (01-12) - year (int): Filter by year (YYYY) - subject_id (int): Filter by subject

Response (200 OK):

```
{
  "success": true,
  "data": {
    "records": [
      {
        "attendance_id": 1001,
        "schedule_id": 50,
        "subject_name": "Data Structures",
        "subject_code": "CS201",
        "teacher_name": "Prof. Smith",
        "date": "2024-01-15",
        "time": "09:00:00",
        "status": "present",
        "verification_status": "success",
        "face_confidence": 92.5,
        "distance_meters": 8.2
      }
    ],
    "pagination": {
      "current_page": 1,
      "total_pages": 3,
      "total_records": 45,
      "per_page": 20
    },
    "summary": {
      "total_classes": 45,
      "attended": 42,
      "percentage": 93.33
    }
  }
}
```

5. Student Schedule

GET /api/student/schedule?day=Monday

Query Parameters: - day (string): Filter by day of week (Monday-Saturday)

Response (200 OK):

```
{
  "success": true,
  "data": {
    "schedules": [
      {
        "schedule_id": 50,
        "day_of_week": "Monday",
        "subject_name": "Data Structures",
        "subject_code": "CS201",
        "teacher_name": "Prof. Smith",

```

```
        "start_time": "09:00",
        "end_time": "10:00",
        "classroom": "101",
        "building": "Building-A",
        "department_name": "Computer Science",
        "semester": 2,
        "section": "A"
    }
}
}
```

6. Student Profile

GET /api/student/profile

Response (200 OK):

```
{
  "success": true,
  "data": {
    "profile": {
      "id": 1,
      "user_id": 123,
      "full_name": "John Doe",
      "email": "john@example.com",
      "phone": "+91-9876543210",
      "roll_number": "CSE-001-2024",
      "department_id": 2,
      "department_name": "Computer Science",
      "semester": 2,
      "section": "A",
      "batch_year": 2024,
      "admission_date": "2023-08-15",
      "face_registered": true,
      "profile_image": "https://sams.edu/uploads/123.jpg"
    }
  }
}
```

7. Update Profile

PUT /api/student/profile

Request:

```
{
  "phone": "+91-9876543211",
  "section": "B"
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Profile updated successfully"
}
```

Teacher Endpoints

1. Teacher Dashboard

GET /api/teacher/dashboard

Response (200 OK):

```
{
  "success": true,
  "data": {
    "profile": {
      "id": 5,
      "user_id": 200,
      "full_name": "Prof. Smith",
      "email": "smith@example.com",
      "phone": "+91-9876543210",
      "employee_id": "EMP-005",
      "department_id": 2,
      "department_name": "Computer Science"
    },
    "subjects": [
      {
        "subject_id": 5,
        "subject_name": "Data Structures",
        "subject_code": "CS201",
        "department_name": "Computer Science",
        "semester": 2,
        "section": "A",
        "student_count": 45
      }
    ],
    "today_schedule": [
      {
        "schedule_id": 50,
        "subject_name": "Data Structures",
        "subject_code": "CS201",
        "start_time": "09:00",
        "end_time": "10:00",
        "classroom": "101",
        "building": "Building-A",
        "semester": 2,
        "section": "A",
        "student_count": 45,
        "is_active": 1
      }
    ],
    "active_session": {
      "session_id": 1001,
      "schedule_id": 50,
      "subject_name": "Data Structures",
      "subject_code": "CS201",
      "started_at": "2024-01-15T09:00:00Z",
      "expected_end": "2024-01-15T10:00:00Z",
      "total_students": 45,
      "present_count": 42,
      "absent_count": 3
    },
    "total_students": 90,
    "classes_today": 2,
    "classes_this_week": 10,
    "avg_attendance": 92.5
  }
}
```

```
}  
}
```

2. Start Class

POST /api/teacher/start-class

Request:

```
{  
  "schedule_id": 50,  
  "latitude": 28.6139,  
  "longitude": 77.2090  
}
```

Response (200 OK):

```
{  
  "success": true,  
  "message": "Class started successfully",  
  "data": {  
    "session_id": 1001,  
    "schedule_id": 50,  
    "subject_code": "CS201",  
    "subject_name": "Data Structures",  
    "started_at": "2024-01-15T09:00:00Z",  
    "expected_end": "2024-01-15T10:00:00Z",  
    "total_students": 45,  
    "present_count": 0,  
    "absent_count": 45,  
    "attendance_window_minutes": 30  
  }  
}
```

3. End Class

POST /api/teacher/end-class

Request:

```
{  
  "session_id": 1001  
}
```

Response (200 OK):

```
{  
  "success": true,  
  "message": "Class ended successfully",  
  "data": {  
    "session_id": 1001,  
    "schedule_id": 50,  
    "subject_code": "CS201",  
    "subject_name": "Data Structures",  
    "total_students": 45,  
    "present_count": 42,  
    "absent_count": 3,  
    "late_count": 0,  
    "not_marked": 0,  
    "attendance_percentage": 93.33,  
  }  
}
```

```

    "ended_at": "2024-01-15T10:00:00Z",
    "auto_marked_absent": 3
  }
}

```

4. Class Attendance Status

GET /api/teacher/class-attendance?schedule_id=50&date=2024-01-15

Query Parameters: - schedule_id (int): Schedule ID - date (string): Date in YYYY-MM-DD format

Response (200 OK):

```

{
  "success": true,
  "data": {
    "schedule_id": 50,
    "subject_name": "Data Structures",
    "subject_code": "CS201",
    "total_students": 45,
    "present_count": 42,
    "absent_count": 3,
    "students": [
      {
        "student_id": 1,
        "student_name": "John Doe",
        "roll_number": "CSE-001-2024",
        "status": "present",
        "marked_at": "2024-01-15T09:05:00Z",
        "face_confidence": 92.5
      }
    ]
  }
}

```

5. Manual Attendance

POST /api/teacher/manual-attendance

Request:

```

{
  "schedule_id": 50,
  "student_id": 1,
  "status": "present"
}

```

Response (200 OK):

```

{
  "success": true,
  "message": "Attendance marked successfully",
  "data": {
    "attendance_id": 1001,
    "student_id": 1,
    "status": "present"
  }
}

```

6. Teacher Schedule

GET /api/teacher/schedule?month=01&year=2024

Query Parameters: - month (int): Month (01-12) - year (int): Year (YYYY)

Response (200 OK):

```
{
  "success": true,
  "data": {
    "schedules": [
      {
        "schedule_id": 50,
        "day_of_week": "Monday",
        "start_time": "09:00",
        "end_time": "10:00",
        "classroom": "101",
        "building": "Building-A",
        "subject_name": "Data Structures",
        "subject_code": "CS201",
        "semester": 2,
        "section": "A",
        "student_count": 45,
        "is_active": 1
      }
    ]
  }
}
```

Admin Endpoints

1. Get All Users

GET /api/admin/users?role=teacher&page=1&limit=20

Query Parameters: - role (string): Filter by user role (admin, teacher, student) - department_id (int): Filter by department - is_active (boolean): Filter by active status - page (int): Page number - limit (int): Records per page

Response (200 OK):

```
{
  "success": true,
  "data": {
    "users": [
      {
        "id": 200,
        "full_name": "Prof. Smith",
        "email": "smith@example.com",
        "role": "teacher",
        "phone": "+91-9876543210",
        "is_active": true,
        "last_login": "2024-01-15T10:00:00Z",
        "created_at": "2023-08-15T08:00:00Z"
      }
    ],
    "pagination": {
      "current_page": 1,

```

```
    "total_pages": 5,  
    "total_records": 100,  
    "per_page": 20  
  }  
}
```

2. Create User (Bulk Upload)

POST /api/admin/users

Request:

```
{  
  "users": [  
    {  
      "full_name": "Raj Kumar",  
      "email": "raj@example.com",  
      "password": "TempPass123",  
      "role": "student",  
      "phone": "+91-9876543210"  
    }  
  ]  
}
```

Response (201 Created):

```
{  
  "success": true,  
  "message": "1 user created, 0 failed",  
  "data": {  
    "created": 1,  
    "failed": 0,  
    "errors": []  
  }  
}
```

3. Update User

PUT /api/admin/users/{user_id}

Request:

```
{  
  "full_name": "Prof. Smith Jr.",  
  "phone": "+91-9876543211",  
  "is_active": true  
}
```

Response (200 OK):

```
{  
  "success": true,  
  "message": "User updated successfully"  
}
```

4. Get Departments

GET /api/admin/departments?department_id=2

Response (200 OK):

```
{
  "success": true,
  "data": {
    "departments": [
      {
        "id": 2,
        "name": "Computer Science",
        "code": "CSE",
        "hod_id": 10,
        "hod_name": "Dr. John",
        "is_active": true
      }
    ]
  }
}
```

5. Get Subjects

GET /api/admin/subjects?department_id=2&semester=2

Query Parameters: - department_id (int): Filter by department - semester (int): Filter by semester

Response (200 OK):

```
{
  "success": true,
  "data": {
    "subjects": [
      {
        "id": 5,
        "name": "Data Structures",
        "code": "CS201",
        "department_id": 2,
        "credits": 3,
        "semester": 2
      }
    ]
  }
}
```

6. Teacher Assignments

GET /api/admin/teacher-assignments?teacher_id=5

Response (200 OK):

```
{
  "success": true,
  "data": {
    "assignments": [
      {
        "assignment_id": 7,
        "teacher_id": 5,
        "teacher_name": "Prof. Smith",
        "subject_id": 5,

```

```

    "subject_name": "Data Structures",
    "subject_code": "CS201",
    "department_id": 2,
    "semester": 2,
    "section": "A",
    "academic_year": "2024-25"
  }
]
}
}

```

7. Attendance Reports

GET /api/admin/reports/attendance?department_id=2&month=01&year=2024

Query Parameters: - department_id (int): Filter by department - month (int): Month (01-12) - year (int): Year (YYYY) - report_type (string): 'student' or 'class'

Response (200 OK):

```

{
  "success": true,
  "data": {
    "report": [
      {
        "student_id": 1,
        "student_name": "John Doe",
        "roll_number": "CSE-001-2024",
        "total_classes": 20,
        "present": 18,
        "absent": 2,
        "percentage": 90.0
      }
    ]
  }
}

```

8. System Settings

GET /api/admin/settings

Response (200 OK):

```

{
  "success": true,
  "data": {
    "settings": {
      "gps_proximity_radius": 50,
      "face_confidence_threshold": 75,
      "late_threshold_minutes": 15,
      "academic_year": "2024-25",
      "semester_current": 2,
      "session_timeout": 3600,
      "maintenance_mode": false
    }
  }
}

```

9. Update Settings

PUT /api/admin/settings

Request:

```
{
  "gps_proximity_radius": 60,
  "face_confidence_threshold": 80,
  "late_threshold_minutes": 20
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Settings updated successfully"
}
```

Common Endpoints

1. Register FCM Token

POST /api/fcm/register-token

Request:

```
{
  "token": "fcm_registration_token_abc123",
  "device_type": "android",
  "device_name": "Samsung Galaxy S21"
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "FCM token registered successfully"
}
```

2. Get Notifications

GET /api/notifications?page=1&limit=20&is_read=false

Query Parameters: - is_read (boolean): Filter by read status - type (string): Filter by notification type - page (int): Page number - limit (int): Records per page

Response (200 OK):

```
{
  "success": true,
  "data": {
    "notifications": [
      {
        "id": 1001,
        "title": "Attendance Alert",

```

```

    "message": "Please mark your attendance for Data Structures class",
    "type": "attendance_alert",
    "is_read": false,
    "created_at": "2024-01-15T09:00:00Z",
    "data": {
      "schedule_id": 50,
      "subject_name": "Data Structures"
    }
  },
  "unread_count": 5,
  "pagination": {
    "current_page": 1,
    "total_pages": 1,
    "total_records": 5
  }
}

```

3. Mark Notification as Read

POST /api/notifications/mark-read

Request:

```

{
  "notification_id": 1001,
  "mark_all": false
}

```

Response (200 OK):

```

{
  "success": true,
  "message": "Notification marked as read"
}

```

4. Logout

POST /api/logout

Response (200 OK):

```

{
  "success": true,
  "message": "Logged out successfully"
}

```

5. Health Check

GET /api/health-check

Response (200 OK):

```

{
  "success": true,
  "status": "operational",

```

```
"database": "connected",  
"firebase": "configured",  
"timestamp": "2024-01-15T10:00:00Z"  
}
```

Error Handling

Validation Errors

Status: 400 Bad Request

```
{  
  "success": false,  
  "message": "Validation failed",  
  "errors": {  
    "email": "Email is required",  
    "password": "Password must be at least 6 characters"  
  }  
}
```

Authentication Errors

Status: 401 Unauthorized

```
{  
  "success": false,  
  "message": "Please login to continue"  
}
```

Authorization Errors

Status: 403 Forbidden

```
{  
  "success": false,  
  "message": "Access restricted to teachers only"  
}
```

Not Found Errors

Status: 404 Not Found

```
{  
  "success": false,  
  "message": "Student profile not found"  
}
```

Server Errors

Status: 500 Internal Server Error

```
{  
  "success": false,  
  "message": "An unexpected error occurred"  
}
```

Status Codes

Code	Meaning	Use Case
200	OK	Successful GET, PUT, POST, DELETE
201	Created	Resource successfully created
204	No Content	Successful DELETE
400	Bad Request	Invalid input, validation failure
401	Unauthorized	Not authenticated, login required
403	Forbidden	Authenticated but not authorized
404	Not Found	Resource not found
409	Conflict	Duplicate record, unique constraint violation
422	Unprocessable Entity	Semantic error (e.g., out of GPS range)
429	Too Many Requests	Rate limit exceeded
500	Server Error	Unhandled exception
503	Service Unavailable	Database down, maintenance mode

Summary

API Design Principles: 1. **RESTful:** Logical resource-based URLs 2. **Consistent:** Standard request/response format 3. **Status-based:** Appropriate HTTP status codes 4. **Error Handling:** Detailed error messages 5. **Pagination:** Large datasets paginated 6. **Role-Based:** Access controlled by user role 7. **Validation:** Input validation before processing

Authentication Flow: 1. POST /api/login → Get session cookie 2. Include session in subsequent requests 3. POST /api/logout → Destroy session

Key Features: - GPS + Face verification for attendance - Real-time notifications via FCM - Role-based access control - Comprehensive error handling - Pagination for large datasets